

## ■ LEVEL 9 • MONOTONIC STACK

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

|                           |                                                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Pattern Trigger</b>    | Next/previous greater or smaller. Remove elements that can no longer become answers.                                    |
| <b>Reference Examples</b> | Next Greater Element I/II • Daily Temperatures • Largest Rectangle in Histogram • Stock Span • Previous Smaller Element |
| <b>Problems</b>           | 6 focused problems in this level                                                                                        |
| <b>Code Standard</b>      | C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings                                   |

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

## LEVEL 9 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

| #  | Problem                        | Pattern                       | LeetCode |
|----|--------------------------------|-------------------------------|----------|
| 1  | Next Greater Element I         | Monotonic Decreasing Stack    | #496     |
| 2  | Next Greater Element II        | Circular Monotonic Stack      | #503     |
| 3  | Largest Rectangle in Histogram | Monotonic Increasing Stack    | #84      |
| 4  | Stock Span                     | Monotonic Stack               | Absent   |
| 5  | Sum of Subarray Minimums       | Monotonic Stack               | #907     |
| 6  | Trapping Rain Water            | Two Pointers / Boundary State | #42      |
| 7  | Next Greater Element I         | Decreasing Monotonic Stack    | 496      |
| 8  | Next Greater Element II        | Circular Monotonic Stack      | 503      |
| 9  | Daily Temperatures             | Monotonic Decreasing Stack    | 739      |
| 10 | Largest Rectangle in Histogram | Monotonic Increasing Stack    | 84       |

**Extension rule:** Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

# PROBLEM 1 Next Greater Element I

Pattern: Monotonic Decreasing Stack

LeetCode: #496

|                       |                                                                        |
|-----------------------|------------------------------------------------------------------------|
| ■ Input               | nums1 and nums2                                                        |
| ■ Output              | next greater value for each nums1 element                              |
| ■ Constraints         | nums1 values appear in nums2                                           |
| ■ Main Idea / Pattern | Scan nums2. Pop smaller unresolved values when a larger value arrives. |
| ■ Time Complexity     | O(n)                                                                   |
| ■ Space Complexity    | O(n)                                                                   |

## ■ Optimal C++ Code

```
vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
    unordered_map<int, int> next;
    stack<int> st;

    for (int x : nums2) {
        while (!st.empty() && x > st.top()) {
            next[st.top()] = x;
            st.pop();
        }
        st.push(x);
    }

    vector<int> ans;
    for (int x : nums1)
        ans.push_back(next.count(x) ? next[x] : -1);

    return ans;
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 2 Next Greater Element II

Pattern: Circular Monotonic Stack

LeetCode: #503

|                       |                                                        |
|-----------------------|--------------------------------------------------------|
| ■ Input               | circular integer array                                 |
| ■ Output              | next greater element for every index                   |
| ■ Constraints         | $n \leq 10^4$                                          |
| ■ Main Idea / Pattern | Scan indices twice. Use a decreasing stack of indices. |
| ■■ Time Complexity    | $O(n)$                                                 |
| ■ Space Complexity    | $O(n)$                                                 |

### ■ Optimal C++ Code

```
vector<int> nextGreaterElements(vector<int>& nums) {  
    int n = nums.size();  
    vector<int> ans(n, -1);  
    stack<int> st;  
  
    for (int i = 0; i < 2 * n; i++) {  
        int idx = i % n;  
  
        while (!st.empty() && nums[idx] > nums[st.top()]) {  
            ans[st.top()] = nums[idx];  
            st.pop();  
        }  
  
        if (i < n)  
            st.push(idx);  
    }  
    return ans;  
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 3 Largest Rectangle in Histogram

Pattern: Monotonic Increasing Stack

LeetCode: #84

|                       |                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------|
| ■ Input               | bar heights                                                                       |
| ■ Output              | largest rectangle area                                                            |
| ■ Constraints         | $1 \leq n \leq 10^5$                                                              |
| ■ Main Idea / Pattern | For each bar, find the first smaller bar on both sides using an increasing stack. |
| ■ Time Complexity     | $O(n)$                                                                            |
| ■ Space Complexity    | $O(n)$                                                                            |

### ■ Optimal C++ Code

```
int largestRectangleArea(vector<int>& h) {
    stack<int> st;
    int ans = 0;
    h.push_back(0);

    for (int i = 0; i < h.size(); i++) {
        while (!st.empty() && h[i] < h[st.top()]) {
            int height = h[st.top()];
            st.pop();

            int left = st.empty() ? -1 : st.top();
            ans = max(ans, height * (i - left - 1));
        }
        st.push(i);
    }

    return ans;
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 4 Stock Span

Pattern: Monotonic Stack

LeetCode: ■ Absent

|                       |                                                                     |
|-----------------------|---------------------------------------------------------------------|
| ■ Input               | daily stock prices                                                  |
| ■ Output              | span of each day                                                    |
| ■ Constraints         | positive prices                                                     |
| ■ Main Idea / Pattern | Remove earlier prices that are less than or equal to today's price. |
| ■■ Time Complexity    | O(n)                                                                |
| ■ Space Complexity    | O(n)                                                                |

### ■ Optimal C++ Code

```
vector<int> stockSpan(vector<int>& price) {  
    stack<int> st;  
    vector<int> ans(price.size());  
    for (int i = 0; i < price.size(); i++) {  
        while (!st.empty() && price[st.top()] <= price[i])  
            st.pop();  
        ans[i] = st.empty() ? i + 1 : i - st.top();  
        st.push(i);  
    }  
    return ans;  
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 5 Sum of Subarray Minimums

Pattern: Monotonic Stack

LeetCode: #907

|                       |                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------|
| ■ Input               | integer array                                                                                    |
| ■ Output              | sum of minimum values over all subarrays                                                         |
| ■ Constraints         | $n \leq 3 * 10^4$                                                                                |
| ■ Main Idea / Pattern | Count how many subarrays use each element as their minimum via previous/next smaller boundaries. |
| ■ Time Complexity     | $O(n)$                                                                                           |
| ■ Space Complexity    | $O(n)$                                                                                           |

### ■ Optimal C++ Code

```
int sumSubarrayMins(vector<int>& arr) {
    const long long MOD = 1e9 + 7;
    int n = arr.size();
    vector<int> left(n), right(n);
    stack<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] > arr[i])
            st.pop();
        left[i] = st.empty() ? i + 1 : i - st.top();
        st.push(i);
    }

    while (!st.empty()) st.pop();

    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && arr[st.top()] >= arr[i])
            st.pop();
        right[i] = st.empty() ? n - i : st.top() - i;
        st.push(i);
    }

    long long ans = 0;
    for (int i = 0; i < n; i++)
        ans = (ans + 1LL * arr[i] * left[i] * right[i]) % MOD;

    return ans;
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 6 Trapping Rain Water

Pattern: Two Pointers / Boundary State

LeetCode: #42

|                       |                                                                         |
|-----------------------|-------------------------------------------------------------------------|
| ■ Input               | bar heights                                                             |
| ■ Output              | total trapped rain water                                                |
| ■ Constraints         | non-negative heights                                                    |
| ■ Main Idea / Pattern | Maintain left/right maxima and move the side with the smaller boundary. |
| ■■ Time Complexity    | O(n)                                                                    |
| ■ Space Complexity    | O(1)                                                                    |

### ■ Optimal C++ Code

```
int trap(vector<int>& h) {
    int l = 0, r = h.size() - 1;
    int leftMax = 0, rightMax = 0;
    int ans = 0;

    while (l < r) {
        if (h[l] <= h[r]) {
            leftMax = max(leftMax, h[l]);
            ans += leftMax - h[l];
            l++;
        } else {
            rightMax = max(rightMax, h[r]);
            ans += rightMax - h[r];
            r--;
        }
    }

    return ans;
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.



## PROBLEM 7 Next Greater Element I

**Pattern:** Decreasing Monotonic Stack

**LeetCode:** 496

|                            |                                                                                 |
|----------------------------|---------------------------------------------------------------------------------|
| <b>Input</b>               | nums1, nums2                                                                    |
| <b>Output</b>              | next greater value for each nums1 element                                       |
| <b>Constraints</b>         | Distinct values; nums1 is a subset of nums2                                     |
| <b>Main Idea / Pattern</b> | Scan nums2 from right and remove smaller elements that can never be the answer. |
| <b>Time Complexity</b>     | O(n)                                                                            |
| <b>Space Complexity</b>    | O(n)                                                                            |

### Optimal C++ Code

```
class Solution {
public:
    vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
        unordered_map<int, int> next;
        stack<int> st;
        for (int i = nums2.size() - 1; i >= 0; i--) {
            while (!st.empty() && st.top() <= nums2[i]) {
                st.pop();
            }
            next[nums2[i]] = st.empty() ? -1 : st.top();
            st.push(nums2[i]);
        }
        vector<int> ans;
        for (int x : nums1) {
            ans.push_back(next[x]);
        }
        return ans;
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.

## PROBLEM 8 Next Greater Element II

**Pattern:** Circular Monotonic Stack

**LeetCode:** 503

|                            |                                                                         |
|----------------------------|-------------------------------------------------------------------------|
| <b>Input</b>               | nums                                                                    |
| <b>Output</b>              | next greater value in a circular array                                  |
| <b>Constraints</b>         | 1 <= nums.size() <= 1e4                                                 |
| <b>Main Idea / Pattern</b> | Traverse twice using $i \% n$ so elements can see values after the end. |
| <b>Time Complexity</b>     | $O(n)$                                                                  |
| <b>Space Complexity</b>    | $O(n)$                                                                  |

### Optimal C++ Code

```
class Solution {
public:
    vector<int> nextGreaterElements(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(n, -1);
        stack<int> st;

        for (int i = 2 * n - 1; i >= 0; i--) {
            int x = nums[i % n];
            while (!st.empty() && st.top() <= x) {
                st.pop();
            }
            if (i < n && !st.empty()) {
                ans[i] = st.top();
            }
            st.push(x);
        }
        return ans;
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.

## PROBLEM 9 Daily Temperatures

**Pattern:** Monotonic Decreasing Stack

**LeetCode:** 739

|                            |                                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------|
| <b>Input</b>               | temperatures                                                                                       |
| <b>Output</b>              | days until a warmer temperature                                                                    |
| <b>Constraints</b>         | 1 <= temperatures.size() <= 1e5                                                                    |
| <b>Main Idea / Pattern</b> | Keep indices whose warmer day has not been found; pop them when the current temperature is larger. |
| <b>Time Complexity</b>     | O(n)                                                                                               |
| <b>Space Complexity</b>    | O(n)                                                                                               |

### Optimal C++ Code

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        int n = temperatures.size();
        vector<int> ans(n, 0);
        stack<int> st;

        for (int i = 0; i < n; i++) {
            while (!st.empty() && temperatures[i] > temperatures[st.top()]) {
                int j = st.top();
                st.pop();
                ans[j] = i - j;
            }
            st.push(i);
        }
        return ans;
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.

## PROBLEM 10 Largest Rectangle in Histogram

Pattern: Monotonic Increasing Stack

LeetCode: 84

|                     |                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------|
| Input               | heights                                                                                 |
| Output              | largest rectangle area                                                                  |
| Constraints         | 1 <= heights.size() <= 1e5                                                              |
| Main Idea / Pattern | For each bar, find the nearest smaller boundaries using an increasing stack of indices. |
| Time Complexity     | O(n)                                                                                    |
| Space Complexity    | O(n)                                                                                    |

### Optimal C++ Code

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        stack<int> st;
        int ans = 0;
        heights.push_back(0);
        for (int i = 0; i < heights.size(); i++) {
            while (!st.empty() && heights[st.top()] > heights[i]) {
                int h = heights[st.top()];
                st.pop();
                int left = st.empty() ? -1 : st.top();
                ans = max(ans, h * (i - left - 1));
            }
            st.push(i);
        }
        return ans;
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.